```
// Vapi -> Google Sheets webhook server
// Receives the "end-of-call-report" from Vapi and writes a clean row to Google S

const express = require("express");
const { google } = require("googleapis");

const app = express();
app.use(express.json({ limit: "5mb" }));

const PORT = process.env.PORT || 3000;
const SPREADSHEET_ID = process.env.SPREADSHEET_ID;
const SHEET_NAME = process.env.SHEET_NAME || "Sheet1";

// Google auth using a service account (set up instructions in README.md)
// Preferred: GOOGLE_CREDENTIALS_BASE64 - the whole service account JSON file, ba
// This avoids all copy-paste issues with newlines/quotes in the private key.
// Fallback: separate GOOGLE_CLIENT_EMAIL + GOOGLE_PRIVATE_KEY variables.
let credentials;
if (process.env.GOOGLE_CREDENTIALS_BASE64) {
  const decoded = Buffer.from(process.env.GOOGLE_CREDENTIALS_BASE64, "base64").to
  const parsed = JSON.parse(decoded);
  credentials = {
    client_email: parsed.client_email,
    private_key: parsed.private_key,
  };
} else {
  credentials = {
    client_email: process.env.GOOGLE_CLIENT_EMAIL,
    private_key: (process.env.GOOGLE_PRIVATE_KEY || "").replace(/\\n/g, "\n"),
  };
}

const auth = new google.auth.GoogleAuth({
  credentials,
  scopes: [
    "https://www.googleapis.com/auth/spreadsheets",
    "https://www.googleapis.com/auth/calendar",
  ],
});

const sheets = google.sheets({ version: "v4", auth });
const calendar = google.calendar({ version: "v3", auth });
const CALENDAR_ID = (process.env.CALENDAR_ID || "").trim();
```

```
// Simple health check so you can confirm the server is alive in a browser
app.get("/", (req, res) => {
  res.send("Vapi webhook server is running.");
});

app.post("/webhook", async (req, res) => {
  try {
    const body = req.body;
    const message = body.message || body; // Vapi nests everything under "message

    // Only act on the final call report - ignore status updates, transcripts, et
    if (message.type !== "end-of-call-report") {
      return res.status(200).send("Ignored (not end-of-call-report)");
    }

    const customer = message.customer || {};
    const analysis = message.analysis || {};
    const call = message.call || {};

    // DEBUG: log the raw analysis object so we can see Vapi's actual field names
    console.log("Raw analysis object:", JSON.stringify(analysis, null, 2));

    // Try a few likely places Vapi puts the caller's name (varies by setup)
    const name =
      analysis.structuredData?.name ||
      customer.name ||
      "Unknown";

    const phone = analysis.structuredData?.phone || customer.number || "Unknown";

    const email =
      analysis.structuredData?.email ||
      "Unknown";

    const reason = analysis.structuredData?.reason || "Unknown";

    const summary = message.summary || analysis.summary || "No summary";

    // Row order matches sheet headers: name | phone | email | reason for call |
    const row = [name, phone, email, reason, summary];

    await sheets.spreadsheets.values.append({
      spreadsheetId: SPREADSHEET_ID,
      range: `${SHEET_NAME}!A:E`,
      valueInputOption: "USER_ENTERED",
      insertDataOption: "INSERT_ROWS",
      requestBody: { values: [row] },
```

```
    });

    console.log("Row added:", row);
    res.status(200).send("OK");
  } catch (err) {
    console.error("Error handling webhook:", err);
    res.status(500).send("Server error");
  }
});

app.listen(PORT, () => {
  console.log(`Server listening on port ${PORT}`);
});

// ----------------------------------------------------------------
// Vapi Tool: check_availability
// Vapi calls this mid-conversation to see what times are free on a given date.
// Expects: { "message": { "toolCalls": [{ "id": "...", "function": { "arguments"
// ----------------------------------------------------------------
app.post("/api/check-availability", async (req, res) => {
  try {
    const toolCall = req.body.message?.toolCalls?.[0];
    const args = toolCall?.function?.arguments || {};
    const date = args.date; // expected format: "YYYY-MM-DD"

    if (!date) {
      return res.status(200).json({
        results: [{ toolCallId: toolCall?.id, result: "No date provided." }],
      });
    }

    // Business hours: 9am - 5pm, 1-hour slots. Adjust to fit the real business.
    const dayStart = new Date(`${date}T09:00:00`);
    const dayEnd = new Date(`${date}T17:00:00`);

    const busy = await calendar.freebusy.query({
      requestBody: {
        timeMin: dayStart.toISOString(),
        timeMax: dayEnd.toISOString(),
        items: [{ id: CALENDAR_ID }],
      },
    });

    const busySlots = busy.data.calendars[CALENDAR_ID].busy || [];

    const freeSlots = [];
    let slot = new Date(dayStart);
```

```
    while (slot < dayEnd) {
      const slotEnd = new Date(slot.getTime() + 60 * 60 * 1000);
      const overlaps = busySlots.some((b) => {
        const bStart = new Date(b.start);
        const bEnd = new Date(b.end);
        return slot < bEnd && slotEnd > bStart;
      });
      if (!overlaps) {
        freeSlots.push(
          slot.toLocaleTimeString("en-GB", { hour: "2-digit", minute: "2-digit" }
        );
      }
      slot = slotEnd;
    }

    const resultText =
      freeSlots.length > 0
        ? `Available times on ${date}: ${freeSlots.join(", ")}`
        : `No availability on ${date}.`;

    res.status(200).json({
      results: [{ toolCallId: toolCall?.id, result: resultText }],
    });
  } catch (err) {
    console.error("Error checking availability:", err);
    res.status(500).json({ error: "Server error" });
  }
});

// ---------------------------------------------------------------
// Vapi Tool: book_appointment
// Vapi calls this once the caller confirms a specific date/time.
// Expects arguments: { date: "2026-07-20", time: "14:00", name: "...", reason: "
// ---------------------------------------------------------------
app.post("/api/book-appointment", async (req, res) => {
  try {
    const toolCall = req.body.message?.toolCalls?.[0];
    const args = toolCall?.function?.arguments || {};
    const { date, time, name, reason } = args;

    if (!date || !time) {
      return res.status(200).json({
        results: [{ toolCallId: toolCall?.id, result: "Missing date or time." }],
      });
    }

    const startTime = new Date(`${date}T${time}:00`);
```

```
    const endTime = new Date(startTime.getTime() + 60 * 60 * 1000);

    await calendar.events.insert({
      calendarId: CALENDAR_ID,
      requestBody: {
        summary: `Appointment - ${name || "Unknown caller"}`,
        description: reason || "Booked via Sophie (AI receptionist)",
        start: { dateTime: startTime.toISOString() },
        end: { dateTime: endTime.toISOString() },
      },
    });

    res.status(200).json({
      results: [
        {
          toolCallId: toolCall?.id,
          result: `Booked for ${date} at ${time}.`,
        },
      ],
    });
  } catch (err) {
    console.error("Error booking appointment:", err);
    res.status(500).json({ error: "Server error" });
  }
});
```